PERSONAL EXPENSE TRACKER

A College Mini-Project Report

A Seminar report submitted
In partial fulfilment of the requirements for the Degree of
Bachelor of Technology

In

Information Technology
Semester – IV

Under the subject of

Programming with Python (202044504)

By

Mahi Patel(12502080603022)

Submitted to : Nayan Mali
Faculty Guide

Academic Year: 2026–27

A D Patel Institute of Technology
New Vallabh Vidyanagar, Anand, Gujarat, India

CERTIFICATE

This is to certify that the mini-project report entitled “Personal Expense Tracker Using Python” has been prepared by **Mahi Patel (12502080603022)** , of the Department of Information Technology, A D Patel Institute of Technology, during the academic year 2026–27.
The report is submitted in partial fulfilment of the academic requirements for the Bachelor of Technology degree. The work described herein concerns a Python-based application intended to record, categorize, and summarize personal income and expenses.
The implementation details in this report are based on the project description supplied for documentation. As the original source code was not supplied for inspection, technology-specific details and sample code identified as illustrative or proposed must be checked against the final application before submission.

Date: _____
Place: _____

Faculty Guide	Head of Department
Nayan Mali Signature: _____	Priyang Bhatt Signature: _____

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to my seminar guide, Prof. Nayan Mali , for
her valuable guidance, continuous support, and encouragement throughout the completion
of this seminar work. His insights into the intersection of technology and social justice
greatly shaped the direction of this report.
I am also thankful to the Head of the Department, Dr. Priyang Bhatt, and all the faculty
members of A D Patel Institute of Technology for providing the necessary facilities and
support. Special thanks to my fellow students for their constructive discussions and feedback.
This report is dedicated to all individuals working tirelessly to combat human trafficking using
technology, law, and compassion.

Name : Mahi Patel
Enrollment No: 125020806030022

ABSTRACT

The Personal Expense Tracker Using Python is a mini-project concept designed to help an individual maintain a structured record of income and expenses. Managing money using memory, paper notes, or scattered spreadsheets can make it difficult to identify spending patterns and calculate the amount remaining after expenses. A simple digital tracker can make these tasks
more organized and repeatable.
The project description specifies a web application with Python used for application logic and text or CSV files used for data storage. The stated functions include adding income and expense records, assigning categories, viewing transaction history, updating or deleting transactions, calculating totals and balance, and validating input. Together, these functions support basic day-to-day personal financial record keeping.
The report explains the problem, requirements, proposed architecture, data flow, module responsibilities, algorithms, testing approach, calculations, limitations, and future improvements. Because the actual source code and dependency list were not supplied, this report does not claim that a particular web framework, chart library, database, or test suite has been verified. Code samples and diagrams are therefore presented as implementation guidance, not as proof of the existing code.
The project is educational in scope. It demonstrates how a programming language can be used to collect structured input, validate data, persist records, perform arithmetic operations, and present useful summaries. It is not intended to replace professional accounting software or financial advice.
Keywords: Python, expense tracking, income, transactions, CSV, file handling, validation, personal finance.

TABLE OF CONTENTS

Section	Page*
Preliminary pages	—
1. Introduction	1
1.1 Background of Expense Management	1
1.2 Need for the Project	1
1.3 Problem Statement	2
1.4 Proposed Solution	2
1.5 Objectives	2
1.6 Scope	2
2. System Analysis	3
2.1 Existing System	3
2.2 Limitations	3
2.3 Proposed System	4
2.4 Functional Requirements	4
2.5 Non-Functional Requirements	5
2.6 Hardware and Software Requirements	5
3. Technology Used	6
3.1 Python	6
3.2 Web Application Context	6
3.3 File Storage	7
3.4 Libraries and Environment	7
4. System Design	8
4.1 Architecture	8
4.2 Modules	8
4.3 DFD Level 0	9
4.4 DFD Level 1	9
4.5 Use-Case Diagram	10
4.6 Flowchart	10
4.7 Data-Storage Design	11
5. Module Description	12
6. Implementation	14
6.1 Folder Structure	14
6.2 Functions	14
6.3 Storage and Retrieval	15
6.4 Calculations and Validation	15
7. Algorithm and Working	16
8. Testing	18
9. Results and Output	20
10. Advantages and Limitations	22
11. Future Scope	23
12. Conclusion	24
13. References	25
14. Appendix	26

CHAPTER 1: INTRODUCTION

1.1 Background of Expense Management

Personal finance management involves understanding how much money is received, where it is spent, and how much remains. Income may come from a salary, allowance, scholarship, freelance work, or other sources. Expenses may include food, transport, education, bills, shopping, and entertainment. When these transactions are recorded consistently, a person can review spending habits and make more informed decisions.
A transaction record usually contains a date, transaction type, category, amount, and description. When records are kept in a consistent format, totals can be calculated automatically and transaction history can be searched or reviewed more easily.

1.2 Need for the Project

Manual tracking is simple at first, but it becomes difficult as the number of transactions increases. A person may forget to record small purchases, miscalculate totals, or find it hard to compare spending across categories. A lightweight application can reduce repetitive arithmetic and provide one place to maintain records.
A web interface can make data entry accessible through a browser. Python can provide the processing logic, while a CSV or text file can provide a simple persistence mechanism for a small educational project. This design is relatively easy to understand, although file-based storage has limitations when several users or concurrent requests are involved.

1.3 Problem Statement

The problem addressed by this project is the lack of a simple, organized way for an individual to record income and expenses, categorize transactions, maintain transaction history, correct or remove records, and calculate overall financial totals. The system should accept understandable input, reject invalid records, and make the stored information available for later review.

1.4 Proposed Solution

The proposed solution is a Python-based web application that accepts transaction details through a user interface. The application validates the supplied data, stores valid transactions in a text or CSV file, and reads the stored records to display history and calculate totals. The precise web framework and front-end implementation are not confirmed because source files were not supplied.

1.5 Project Objectives

- Provide a consistent method for entering income and expense transactions.
- Allow transactions to be grouped into meaningful categories.
- Display transaction history for review.
- Support editing and deletion of previously recorded transactions.
- Calculate total income, total expenses, and remaining balance.
- Validate required fields and numeric amounts before saving.
- Demonstrate practical use of Python logic and file handling in a web application.

1.6 Scope of the Project

The stated scope is basic personal transaction management for a single user or a small demonstration environment. It includes transaction entry, categories, history, record maintenance, calculations, and input validation. Authentication, cloud synchronization, bank integration, automatic transaction imports, budgets, notifications, and mobile applications are not confirmed as implemented and are treated as future possibilities.

CHAPTER 2: SYSTEM ANALYSIS

2.1 Existing System

An existing or manual approach may use a notebook, calculator, spreadsheet, or separate notes to track spending. These approaches can be useful for small volumes of data, but they depend on the user maintaining a consistent process. The project proposes consolidating transaction entry and calculations in a single application.

2.2 Limitations of the Existing Approach

- Manual calculations may contain arithmetic errors.
- Records may be incomplete if transactions are not entered promptly.
- Finding an older transaction can require searching multiple pages or files.
- Category-wise totals require additional effort unless formulas or scripts are maintained.
- Repeated data entry may lead to inconsistent category names or date formats.
- Paper records are not automatically backed up and may be lost or damaged.

2.3 Proposed System

The proposed application uses a structured transaction format and centralized processing logic. After input validation, each transaction is stored as a record. The system can read those records to display history and calculate totals. Editing or deleting a record requires locating it reliably, which is why a unique transaction identifier is recommended even if the original implementation uses a simpler format.

Area	Proposed behaviour	Verification note
Input	Accept transaction type, date, category, amount, and optional description.	Field names should match the actual UI.
Validation	Check required values, allowed type, date, and positive numeric amount.	Exact validation rules depend on source code.
Persistence	Write valid records to a text or CSV file.	CSV is the assumed structured option, not source-verified.
History	Read and present saved transactions.	Filtering and sorting are not confirmed.
Summary	Sum income and expenses and calculate balance.	Formula is defined in Chapter 7.
Maintenance	Update or remove an existing record.	Actual editing/deletion method must be verified.

2.4 Functional Requirements

ID	Requirement	Priority
FR-01	The system shall accept a new income transaction.	High
FR-02	The system shall accept a new expense transaction.	High
FR-03	The system shall associate a category with a transaction.	High
FR-04	The system shall display saved transaction history.	High
FR-05	The system shall allow an existing transaction to be updated.	High
FR-06	The system shall allow a transaction to be deleted.	High
FR-07	The system shall calculate total income.	High
FR-08	The system shall calculate total expenses.	High
FR-09	The system shall calculate remaining balance.	High

FR-10	The system shall validate input and communicate errors.	High

2.5 Non-Functional Requirements

Requirement	Explanation
Usability	Forms and labels should be clear, and validation messages should explain how to correct an error.
Correctness	Totals should be calculated from stored records using consistent rules.
Reliability	A failed write should not silently appear as a successful save.
Maintainability	Transaction logic, storage logic, and interface logic should be separated where practical.
Performance	For a small personal dataset, reading a CSV file should be adequate; performance should be tested with larger files.
Privacy	Financial records should not be exposed publicly. Deployment permissions and access control must be considered.
Portability	Python and text-based data formats can be portable, subject to dependencies and file paths.

2.6 Hardware and Software Requirements

Type	Minimum / suggested requirement
Hardware	Computer capable of running a supported Python version and a modern browser.
Processor	Any recent dual-core processor suitable for basic development.
Memory	4 GB RAM or more recommended for a comfortable development environment.
Storage	A small amount of free disk space for source files, dependencies, and CSV data.
Operating system	Windows, macOS, or Linux, depending on Python and framework compatibility.
Language	Python; exact version to be confirmed from the project.
Browser	A current browser such as Chrome, Edge, Firefox, or Safari.
Editor	VS Code or another Python-compatible editor; not mandatory at runtime.

CHAPTER 3: TECHNOLOGY USED

3.1 Python

Python is a high-level, general-purpose programming language known for readable syntax and a broad standard library. It supports functions, modules, exception handling, and file operations, making it suitable for teaching small applications. In this project concept, Python is used for validation, transaction processing, arithmetic, and file access. The exact role of Python in the web stack must be checked against the source code.

3.2 Web Application Context

A web application normally contains a browser-facing interface and a server-side component. The browser collects user input and displays results. The server receives requests, validates the data, performs calculations, and reads or writes stored records. The project description says that the application is web-based and uses Python, but it does not identify a framework. Flask, Django, and FastAPI are s of Python web frameworks; none is asserted here as the framework used by the submitted project.

3.3 Text or CSV File Storage

A plain text file stores characters in a readable format. A CSV (comma-separated values) file stores tabular records as rows and fields, and can be inspected using a text editor or spreadsheet program. For transaction data, CSV is generally easier to process consistently than free-form text because fields can be named and rows can follow a defined structure.
A suggested CSV header is: transaction_id,date,type,category,amount,description. The amount should be stored in a consistent numeric format. The description may contain punctuation, so a CSV library should be used rather than manually joining strings with commas.

3.4 Libraries and Development Environment

The dependency list was not supplied, so the libraries used by the actual project cannot be confirmed. The following are possible choices, not claims about the implementation:

Library / tool	Possible purpose	Status in this report
csv (Python standard library)	Read and write CSV rows safely.	Suggested if CSV is used.
datetime (standard library)	Parse and validate dates.	Suggested.
decimal (standard library)	Represent currency amounts more safely than binary floating-point in financial calculations.	Suggested.
pathlib (standard library)	Manage file paths in a portable way.	Suggested.
Flask / Django / FastAPI	Expose Python functionality through web routes.	Framework not confirmed.
HTML, CSS, JavaScript	Build the browser interface.	Likely web technologies, but not source-verified.
pytest / unittest	Automate tests.	Not confirmed as used.

3.5 Development Environment

A typical development setup consists of Python, a code editor, a browser, a virtual environment, and a dependency file. A virtual environment helps isolate the packages required by one project. The exact setup instructions should be updated after the actual requirements file and source tree are available.

CHAPTER 4: SYSTEM DESIGN

4.1 System Architecture

The diagram below presents a generic architecture for the described web application. It separates the browser interface, Python application logic, and file storage. It is a proposed logical model and must be adjusted if the actual application uses a different structure.

```
flowchart TD
    U[User] --> B[Web Browser / User Interface]
    B -->|Form request| A[Python Web Application]
    A --> V[Input Validation]
    V -->|Valid data| S[Transaction Service]
    V -->|Invalid data| E[Validation Error Response]
    S <--> F[(CSV / Text File)]
    S --> C[Summary Calculations]
    C --> A
    A -->|HTML / response| B
    E --> B
```

Figure 4.1: Proposed logical architecture.

4.2 Module Description

Module	Responsibility	Input / output
Transaction Entry	Collect income or expense details and submit them.	Input form → transaction request.
Validation	Check required fields and acceptable values.	Request → valid record or error.
Category Management	Associate a transaction with a category.	Transaction type/category.
Transaction History	Retrieve and display stored records.	Stored rows → history view.
Record Maintenance	Find and update or delete a selected record.	Transaction ID → changed file.
Summary Calculator	Compute totals and balance.	Transactions → summary values.
File Storage	Read and write records.	Structured records ↔ file.
Error Handling	Return meaningful errors for invalid data or storage failures.	Exception / invalid request → message.

4.3 Data Flow Diagram — Level 0

A Level 0 DFD, also called a context diagram, treats the whole application as one process. The user supplies transaction details and requests history or summaries. The application returns confirmation, records, or calculated totals. The file is shown as a storage boundary in the proposed model.

```
flowchart LR
    U[User] -->|Add / edit / delete transaction; request summary| P((0: Personal Expense Tracker))
    P -->|Confirmation, history, totals, errors| U
    P <--> D[(D1: Transaction File)]
```

.

Figure 4.2: DFD Level 0.

4.4 Data Flow Diagram — Level

The Level 1 DFD decomposes the application into smaller processes. Validated input is passed to the transaction-management process. The process reads or updates the transaction file. The reporting process uses the stored records to calculate totals.

```
flowchart TD
    U[User] -->|Transaction form| P1((1.0 Validate Input))
    P1 -->|Invalid values| U
    P1 -->|Valid transaction| P2((2.0 Manage Transactions))
    P2 <--> D[(D1: Transaction File)]
    U -->|History request| P3((3.0 Retrieve History))
    P3 <-->|Read records| D
    P3 -->|Transaction list| U
    U -->|Summary request| P4((4.0 Calculate Summary))
    P4 <-->|Read transactions| D
    P4 -->|Income, expenses, balance| U
```

4.5 Use-Case Diagram

The user is the primary actor. The use cases represent the stated project functions. This diagram does not imply the presence of authentication or multiple user roles.

```
flowchart LR
    U[User]
    subgraph PET[Personal Expense Tracker]
      UC1([Add income])
      UC2([Add expense])
      UC3([Categorize transaction])
      UC4([View history])
      UC5([Update transaction])
      UC6([Delete transaction])
      UC7([View totals and balance])
      UC8([Receive validation feedback])
    end
    U --- UC1
    U --- UC2
    U --- UC3
    U --- UC4
    U --- UC5
    U --- UC6
    U --- UC7
    U --- UC8
```

Figure source: Mermaid text is reproducible in a Mermaid-compatible editor. Replace or redraw it if the final implementation differs.

Figure 4.4: Use-case diagram.

4.6 Application Flowchart

```
flowchart TD
    A([Start]) --> B[Open application]
    B --> C[Choose action]
    C --> D{Action?}
    D -->|Add| E[Enter transaction details]
    E --> F{Input valid?}
    F -->|No| G[Show error and request correction]
    G --> E
    F -->|Yes| H[Save transaction]
    D -->|History| I[Read and display records]
    D -->|Summary| J[Read records and calculate totals]
    D -->|Edit / delete| K[Select existing transaction]
    K --> L[Validate and apply change]
    H --> M[Show result]
    I --> M
    J --> M
    L --> M
    M --> N{Another action?}
    N -->|Yes| C
    N -->|No| O([End])
```

4.7 Data-Storage Design

For reliable record maintenance, each transaction should have a unique identifier. The following schema is a recommended CSV design. It is not a verified copy of the project's current file.

Field	Type / format	Purpose	Validation suggestion
transaction_id	String / UUID	Unique key used to identify a record.	Must not be blank or duplicated.
date	YYYY-MM-DD	Date of the transaction.	Must parse as a valid date.
type	income or expense	Determines how the amount contributes to totals.	Restrict to allowed values.
category	Text	Groups transactions, e.g. Food or Salary.	Required; trim whitespace.
amount	Decimal numeric string	Transaction value.	Must be greater than zero.
description	Text	Optional details.	Apply length limits; escape output in HTML.

File-based storage is simple, but it requires care. If a file is overwritten while an update is in progress, a failure could damage data. For safer updates, an implementation can write a complete updated dataset to a temporary file and then replace the original file. Concurrent writes and multi-user access are better handled by a database.

CHAPTER 5: MODULE DESCRIPTION

5.1 Add Income and Expenses

This module captures the basic details of a financial transaction. The transaction type indicates whether money was received or spent. A useful form includes date, type, category, amount, and description. After submission, the server should validate the values before attempting to store the record.
: a monthly allowance of ₹5,000 is recorded as income, while a transport purchase of ₹120 is recorded as an expense. Both records can use the same general transaction structure; the type field determines how they are included in summaries.

5.2 Categorize Transactions

Categories make transaction history easier to understand. income categories include Salary, Allowance, Scholarship, and Other Income. Expense categories may include Food, Transport, Education, Bills, Shopping, Health, and Entertainment. The application should use consistent category labels to avoid splitting the same type of expense across slightly different spellings.
A fixed list of categories is easier to validate, while a user-defined category list is more flexible. The project description confirms categorization but does not specify which approach is implemented.

5.3 View Transaction History

The history module displays previously stored records in a table or list. Useful columns include date, type, category, amount, and description. A clear interface should distinguish income from expenses using labels as well as colour, so meaning does not depend on colour alone. Sorting, filtering, pagination, and date-range selection are optional enhancements unless verified in the actual application.

5.4 Update or Delete Transactions

Users sometimes enter the wrong date, amount, or category. Updating allows the existing record to be corrected without creating a duplicate. Deletion removes a record that should no longer be included. A unique transaction ID is recommended to identify the intended row, because row numbers can change when the file is sorted or rewritten.
A delete action should ask for confirmation where practical. Both update and delete operations should report success only after the storage operation has completed. If a record cannot be found, the application should show an informative message.

5.5 Calculate Total Income, Expenses, and Balance

The summary module processes saved transactions. Total income is the sum of amounts whose type is income. Total expenses is the sum of amounts whose type is expense. Remaining balance is total income minus total expenses. The same stored dataset should be used for both history and summary calculations to avoid inconsistent results.

5.6 Generate Summaries and Reports

The confirmed description includes calculation of totals and balance. A separate downloadable report, monthly report, category breakdown, or export function is not confirmed. Such features should only be described as implemented if the source code demonstrates them. A basic summary can show the total income, total expenses, and resulting balance.

5.7 Visualize Spending Patterns

Charts can help users compare categories or view expenses over time, but chart visualization was not selected as a confirmed implemented feature. It is therefore treated as optional. If later added, a bar chart could compare expense categories and a line chart could show spending by month. Any chart should be generated from the same validated transaction data as the numeric summary.

5.8 Input Validation and Error Handling

Validation protects the data from incomplete or invalid entries. The application should check that required fields are present, the type is one of the allowed values, the date is valid, and the amount is a positive number. It should handle missing files, malformed CSV rows, permission errors, and unexpected failures without exposing internal stack traces to ordinary users.

Situation	Recommended behaviour
Amount is blank	Reject the transaction and request an amount.
Amount contains letters	Reject it with a clear numeric-input message.
Amount is zero or negative	Reject it if the rule requires positive transaction values.
Date is invalid	Ask the user to enter a valid date.
Unknown transaction ID	Report that the record could not be found.
Storage file is unavailable	Show a safe error and avoid claiming the write succeeded.
CSV row is malformed	Handle the row deliberately; do not silently calculate incorrect totals.

CHAPTER 6: IMPLEMENTATION

6.1 Suggested Project Folder Structure

Because the original project files were not supplied, the following is a suggested structure for a small Python web application. It is not an assertion that the existing project uses these filenames or framework.

```
personal-expense-tracker/
├── app.py                  # Application entry point ()
├── services/
│   ├── transactions.py     # Validation and transaction operations
│   └── summaries.py        # Total and balance calculations
├── storage/
│   └── transactions.csv    # Data file created at runtime
├── templates/              # HTML templates, if server-rendered
├── static/
│   ├── css/
│   └── js/
├── tests/                  # Automated tests, if added
├── requirements.txt        # Dependencies, if used
└── README.md               # Setup and usage instructions
```

6.2 Transaction Model

The record model below is a plain Python dictionary for illustration. A real application may use a class, dataclass, ORM model, or framework-specific form object.

```
transaction = {
    "transaction_id": "T-1001",
    "date": "2026-10-25",
    "type": "expense",
    "category": "Transport",
    "amount": "120.00",
    "description": "Bus fare"
}
```

6.3 Validation Function

This sample uses the Decimal type to avoid common binary floating-point representation surprises in money calculations. It checks the type, amount, and required fields. It is illustrative code and has not been run against the user's project.

```
from decimal import Decimal, InvalidOperation
from datetime import date

ALLOWED_TYPES = {"income", "expense"}

def validate_transaction(data):
    required = ("date", "type", "category", "amount")
    for field in required:
        if not str(data.get(field, "")).strip():
            raise ValueError(f"{field} is required")

    tx_type = str(data["type"]).strip().lower()
    if tx_type not in ALLOWED_TYPES:
        raise ValueError("type must be income or expense")

    try:
        date.fromisoformat(str(data["date"]).strip())
    except ValueError as exc:
        raise ValueError("date must use YYYY-MM-DD format") from exc

    try:
        amount = Decimal(str(data["amount"]).strip())
    except InvalidOperation as exc:
        raise ValueError("amount must be numeric") from exc

    if not amount.is_finite() or amount <= 0:
```

```
        raise ValueError("amount must be greater than zero")

    return {
        **data,
        "type": tx_type,
        "category": str(data["category"]).strip(),
        "amount": format(amount, "f"),
    }
```

6.4 CSV Storage

The csv module handles quoting and delimiters more reliably than manually joining values with commas. The below appends a validated transaction to a CSV file. Production code should also manage the file path, headers, permissions, concurrency, and failure recovery.

```
import csv
from pathlib import Path

FIELDS = [
    "transaction_id", "date", "type",
    "category", "amount", "description"
]
DATA_FILE = Path("storage/transactions.csv")

def append_transaction(transaction):
    DATA_FILE.parent.mkdir(parents=True, exist_ok=True)
    needs_header = not DATA_FILE.exists() or DATA_FILE.stat().st_size == 0

    with DATA_FILE.open("a", newline="", encoding="utf-8") as file:
        writer = csv.DictWriter(file, fieldnames=FIELDS)
        if needs_header:
            writer.writeheader()
        writer.writerow({key: transaction.get(key, "") for key in FIELDS})
```

6.5 Summary Calculation

```
from decimal import Decimal

def calculate_summary(transactions):
    total_income = Decimal("0.00")
    total_expenses = Decimal("0.00")

    for tx in transactions:
        amount = Decimal(str(tx["amount"]))
        if tx["type"] == "income":
            total_income += amount
        elif tx["type"] == "expense":
            total_expenses += amount

    return {
        "total_income": total_income,
        "total_expenses": total_expenses,
        "balance": total_income - total_expenses,
    }
```

The function loops through every record once. Each income amount contributes to total income, and each expense amount contributes to total expenses. The final balance is calculated by subtraction. For n transactions, the calculation takes O(n) time and uses O(1) additional numeric accumulator space, excluding the input records already held in memory.

6.6 Update and Delete Logic

For a CSV implementation, update and delete operations usually require reading the rows, finding the record by its unique ID, and writing the revised set of rows. This is more complex than appending a new record because the file must be rewritten safely.

6.7 Exception Handling and Security Considerations

Input validation and exception handling serve different purposes. Validation checks whether the request follows the rules. Exception handling deals with failures that occur while the program runs, such as an unavailable file or an unexpected conversion error. Both are necessary.

- Use parameterized database queries if the storage layer later changes to a database.
- Escape user-provided text before inserting it into HTML to reduce cross-site scripting risk.
- Do not expose file system paths, stack traces, or secret configuration values in user-facing errors.
- Restrict access to the CSV file and avoid placing personal financial records in a public web directory.
- Use authentication and authorization before deploying a multi-user version.
- Do not treat a local CSV file as safe for concurrent writes by multiple users.

CHAPTER 7: ALGORITHM AND WORKING

7.1 Main Algorithm

1. Start the application and load its configuration.
2. Read the requested action from the user interface.
3. If the action adds a transaction, collect the date, type, category, amount, and description.
4. Validate all required fields and the numeric amount.
5. If validation fails, return a clear message and do not save the record.
6. If validation succeeds, assign a unique transaction ID and store the record.
7. For history requests, read valid records and display them.
8. For update or delete requests, locate the selected record and safely write the revised dataset.
9. For summary requests, sum income records and expense records separately.
10. Calculate balance as total income minus total expenses.
11. Display the requested result and allow another action.

7.2 Application Workflow

The application follows a repeated request-response workflow. A user chooses an action, enters data or requests a view, and receives either a validation message or a result. Persistence allows records to remain available after the application is restarted, provided the data file remains intact and the application uses the same file path.

7.3 Transaction Records

The following records are fictional s for explaining the calculations. They are not actual user data or output captured from a running application.

ID	Date	Type	Category	Amount (₹)	Description
T-1001	2026-10-01	Income	Allowance	5,000.00	Monthly allowance
T-1002	2026-10-02	Expense	Food	250.00	Lunch
T-1003	2026-10-04	Expense	Transport	120.00	Bus fare
T-1004	2026-10-05	Income	Freelance	1,500.00	Small assignment
T-1005	2026-10-06	Expense	Education	800.00	Study material
T-1006	2026-10-08	Expense	Entertainment	300.00	Movie ticket

7.4 Calculation of Totals

Total income = ₹5,000.00 + ₹1,500.00 = ₹6,500.00
Total expenses = ₹250.00 + ₹120.00 + ₹800.00 + ₹300.00 = ₹1,470.00
Remaining balance = Total income − Total expenses = ₹6,500.00 − ₹1,470.00 = ₹5,030.00

Summary item	Calculation	result
Total income	Sum of amounts where type = income	₹6,500.00
Total expenses	Sum of amounts where type = expense	₹1,470.00
Remaining balance	Income − expenses	₹5,030.00

7.5 Complexity Discussion

If the application reads n transactions and processes each once, total calculation requires O(n) time. The arithmetic uses a fixed number of accumulators, so the additional working space is O(1). Reading all records into a list uses O(n) memory. A CSV-based update or deletion also generally requires O(n) time and a rewrite of the file. These are analytical expectations for the proposed approach, not benchmark results.

CHAPTER 8: TESTING

8.1 Test Plan

Testing should check that valid records are accepted, invalid records are rejected, history reflects stored data, editing and deletion affect the correct transaction, and totals match independently calculated values. Since no source code or running application was supplied, the table below is a test plan with expected results; the actual-output and status columns are intentionally marked Not Run.

8.2 Test Cases

Test ID	Input / action	Expected output	Actual output	Status
TC-01	Add income of ₹5,000 with valid date and category.	Record saved and appears in history.	Not run	Pending
TC-02	Add expense of ₹250 with valid fields.	Record saved as expense.	Not run	Pending
TC-03	Submit with amount blank.	Validation message; no record saved.	Not run	Pending
TC-04	Submit amount as 'abc'.	Numeric validation error; no record saved.	Not run	Pending
TC-05	Submit amount as zero.	Rejected if positive amount is required.	Not run	Pending
TC-06	Submit negative amount.	Rejected if positive amount is required.	Not run	Pending
TC-07	Submit invalid date such as 2026-13-40.	Date validation message.	Not run	Pending
TC-08	Submit missing category.	Validation message; no record saved.	Not run	Pending
TC-09	View history after adding two records.	Both saved records are displayed.	Not run	Pending
TC-10	Update amount for an existing ID.	Updated amount appears in history and totals.	Not run	Pending
TC-11	Delete an existing transaction.	Record is removed and totals are recalculated.	Not run	Pending
TC-12	Update an ID that does not exist.	Not-found message; other records unchanged.	Not run	Pending
TC-13	Calculate totals for sample records in §7.3.	Income ₹6,500; expenses ₹1,470; balance ₹5,030.	Not run	Pending
TC-14	Start with an empty data file.	Empty history and zero totals, or documented empty-state message.	Not run	Pending
TC-15	Data file is missing or unreadable.	Safe error or controlled file initialization; no false success.	Not run	Pending
TC-16	Description contains a comma or quote.	CSV fields remain correctly separated and readable.	Not run	Pending
TC-17	Repeatedly add multiple records.	No record is silently overwritten or duplicated unintentionally.	Not run	Pending
TC-18	Amount has many decimal places.	Input is rounded or rejected according to documented currency rules.	Not run	Pending

8.3 Testing Approach

- Unit testing: test validation and summary calculation functions independently.
- Integration testing: verify that valid form submissions are written to the file and then returned in history.

- Boundary testing: test empty values, zero, negative values, long descriptions, and decimal precision.
- Error testing: test missing files, permission errors, malformed rows, and unknown transaction IDs.
- Regression testing: repeat previously passed tests after code changes.

8.4 Acceptance Criteria

The project may be considered ready for demonstration when the stated core functions work consistently, valid transactions persist, invalid data is rejected with understandable feedback, history agrees with the stored file, updates and deletions affect only the intended record, and all summary values match hand-calculated results. These are acceptance criteria, not a claim that the current application has passed them.

CHAPTER 10: ADVANTAGES AND LIMITATIONS

10.1 Advantages

- Organizes income and expenses in a consistent structure.
- Reduces the need to calculate totals manually.
- Allows transaction history to be reviewed in one place.
- Categories help users understand where money is spent.
- Editing and deletion provide a way to correct data-entry mistakes.
- Python and CSV/text files are approachable technologies for a learning project.
- A web interface can be accessed through a browser in the configured environment.

10.2 Limitations

- Text or CSV storage has limited support for concurrent access and multi-user use.
- A local file may be lost if it is deleted, corrupted, or not backed up.
- CSV files do not provide database transactions, relational constraints, or built-in access control.
- Bank account synchronization and automatic transaction imports are not included in the confirmed scope.
- Charts, budgets, authentication, and downloadable reports are not confirmed as implemented.
- Financial records may be exposed if the web application is deployed without appropriate access controls.
- Performance with large files has not been measured.

10.3 Risk Mitigation

Risk	Possible mitigation
Accidental file loss	Keep backups and provide a documented restore procedure.
Incorrect totals	Use consistent transaction types, decimal arithmetic, and automated tests.
Duplicate records	Assign unique transaction IDs and prevent accidental duplicate submissions.
Unauthorized access	Add authentication and restrict deployment access before public hosting.
Concurrent write conflicts	Use safe file replacement for a single-user prototype; migrate to a database for multi-user deployment.
Malformed data	Validate rows while reading and report skipped or invalid records explicitly.

CHAPTER 11: FUTURE SCOPE

11.1 User Authentication

A future version can support registration, login, password reset, and session management. Passwords should be hashed using an appropriate password-hashing algorithm and should never be stored as plain text. Each user's transactions must be separated and protected by server-side authorization.

11.2 Cloud Synchronization

Cloud storage could allow users to access their records across devices. This would require a secure hosted backend, authentication, backup procedures, and a clear privacy policy. Simply placing a CSV file in a publicly accessible folder would not be a safe cloud design.

11.3 Budget Alerts

Users could set monthly or category-specific budgets. The system could compare recorded expenses with the selected limit and display a warning when spending approaches or exceeds it. Alerts should be based on the user's chosen rules and should not be described as bank-verified balances.

11.4 Export to CSV or PDF

If the application stores data internally in another format in a future version, export features could produce CSV for spreadsheet analysis and PDF for printable summaries. Exported files should include the reporting period, currency, and generation date.

11.5 Advanced Spending Analytics

Possible analytics include monthly comparisons, category percentages, recurring expense detection, and a spending trend chart. Such analysis should be based on sufficiently complete records and clearly state the period covered. Predictions, if introduced, should be labelled estimates rather than facts.

11.6 Mobile Application Integration

A mobile-friendly responsive interface or a dedicated mobile client could make data entry more convenient. A mobile client would need a secure API, authentication, and a strategy for offline changes and synchronization conflicts.

11.7 Database Migration

If the number of users or records increases, the application could migrate from CSV storage to SQLite or a server database such as PostgreSQL. A database would improve querying, indexing, data constraints, and concurrent access. This is a future option and is not claimed as part of the present project.

CHAPTER 12: CONCLUSION

The Personal Expense Tracker Using Python addresses a common organizational problem: maintaining an accurate record of personal income and expenses. The described core functions cover transaction entry, categorization, history, record correction, deletion, validation, and the calculation of income, expenses, and remaining balance.

The report demonstrates how a small web application can separate user input, validation, processing, and persistence. It also explains the role of a structured text or CSV file, the importance of unique transaction identifiers, and the need to test valid and invalid scenarios. The calculations show how transaction records can be converted into a simple summary.

The project is suitable as an educational mini-project because it connects programming fundamentals with a practical everyday use case. However, the actual source code was not available during report preparation. For that reason, the proposed diagrams, code snippets, dependency suggestions, and test cases must be checked against the completed application before they are presented as implementation evidence. Genuine screenshots and actual test results should be inserted before final academic submission.

CHAPTER 13: REFERENCES

The following official documentation is relevant to the suggested implementation. The specific framework reference should be retained only if that framework is actually used.
[1] Python Software Foundation, “Python 3 Documentation.” https://docs.python.org/3/
[2] Python Software Foundation, “csv — CSV File Reading and Writing.” https://docs.python.org/3/library/csv.html
[3] Python Software Foundation, “decimal — Decimal fixed-point and floating-point arithmetic.” https://docs.python.org/3/library/decimal.html
[4] Python Software Foundation, “datetime — Basic date and time types.” https://docs.python.org/3/library/datetime.html
[5] Python Software Foundation, “pathlib — Object-oriented filesystem paths.” https://docs.python.org/3/library/pathlib.html
[6] Python Software Foundation, “unittest — Unit testing framework.” https://docs.python.org/3/library/unittest.html
[7] Mermaid, “Mermaid Documentation.” https://mermaid.js.org/ — for rendering the reproducible diagram source included in this report.
[8] Flask Documentation. https://flask.palletsprojects.com/ — consult only if Flask is confirmed as the web framework.
[9] Django Documentation. https://docs.djangoproject.com/ — consult only if Django is confirmed as the web framework.
[10] FastAPI Documentation. https://fastapi.tiangolo.com/ — consult only if FastAPI is confirmed as the web framework.
Accessed: 26 October 2026. Verify current documentation URLs and remove references for tools not used in the final project.

CHAPTER 14: APPENDIX

14.1 Appendix A — Sample CSV Data

```
transaction_id,date,type,category,amount,description
T-1001,2026-10-01,income,Allowance,5000.00,Monthly allowance
T-1002,2026-10-02,expense,Food,250.00,Lunch
T-1003,2026-10-04,expense,Transport,120.00,Bus fare
T-1004,2026-10-05,income,Freelance,1500.00,Small assignment
T-1005,2026-10-06,expense,Education,800.00,Study material
T-1006,2026-10-08,expense,Entertainment,300.00,Movie ticket
```

The sample is fictional and intended for demonstration only. The actual CSV header and data types may differ.

14.2 Appendix B — How to Run the Application

Because the source code and framework are not available, the steps below are a general checklist rather than exact run commands for the current project.

12. Obtain the final project folder and inspect its README and dependency file.
13. Install a compatible Python version and create a virtual environment.
14. Install only the dependencies listed by the project.
15. Confirm the configured data-file path and ensure the application has permission to create or update it.
16. Run the documented application entry point (for , a framework-specific development command). Do not use a guessed command if the framework is unknown.
17. Open the local URL printed by the application in a browser.
18. Add fictional test records and verify history and summary calculations.
19. Run the test cases in Chapter 8 and record the actual output and pass/fail status.
20. Capture genuine screenshots for the result section and remove any private financial data.

14.3 Appendix C — Viva Preparation Questions

Question	Key points for an answer
What is the purpose of the project?	To record income and expenses, categorize them, and calculate totals and balance.
Why use Python?	Readable syntax, useful standard library, and support for validation and file handling.
Why use CSV or text storage?	Simple, readable, and suitable for a small prototype; less suitable for concurrent multi-user use.
How is balance calculated?	Total income minus total expenses.
Why is validation important?	It prevents missing or malformed values from entering stored records.
How can a transaction be updated safely?	Locate it by a stable unique ID, validate changes, write the revised data safely, and confirm success.
What is a limitation of CSV storage?	It lacks database transactions and robust concurrent-write control.
What would you improve next?	Authentication, database storage, backups, budgets, exports, analytics, and responsive/mobile improvements.

14.4 Final Submission Checklist

- Replace all bracketed student and roll-number placeholders.
- Confirm the college, department, guide designation, and academic-year wording with the department's format.
- Verify the actual framework, libraries, file structure, and CSV schema from the source code.
- Remove any proposed code or diagrams that do not match the implementation.
- Insert genuine screenshots with figure numbers and captions.
- Execute tests and replace “Not run” with actual output and status only after testing.
- Update the table of contents and page numbers after final layout.
- Check spelling, margins, signatures, and required institutional formatting.